Business Case GoCase

Construção do Banco de Dados e Estruturação

Partes 5 e 6 — Documentação, Justificativas e Requisitos Adicionais

Abril de 2026

Repositório: github.com/intelligent-life-paradox/business_case_gocase

Dashboard: businesscasegocase-jluca.streamlit.app

Dataset: Kaggle — Olist Brazilian E-Commerce

Sumário

Parte 5 — Documentação e Justificativas **2**

1 Escolha e Origem dos Dados **2**

2 Modelagem do Banco de Dados **2**

2.1 Escolha do SQLite 2

2.2 Schema e Normalização — Terceira Forma Normal (3FN) 2

2.3 Escolha da Ferramenta de Visualização: Streamlit 3

3 Lógica das Consultas SQL **3**

3.1 Por que Evitar Table Scans — O Problema $O(n)$ 3

3.2 Consulta 1 — Faturamento Mensal por Categoria 4

3.3 Consulta 2 — Produto Mais Vendido por Filial 5

3.4 Consulta 3 — Ticket Médio e LTV por Cliente 5

3.5 Consulta 4 — Clientes Fiéis por Região 6

3.6 Consulta 5 — Tendências Regionais por Mês 6

Parte 6 — Requisitos Adicionais **8**

4 Google Sheets com JavaScript — Gestão de Limites e Quotas **8**

4.1 Controle de Rate Limiting com Back-off Exponencial 8

4.2 Batching — Nunca Uma Requisição por Célula 8

4.3 Paralelismo Controlado e Caching 8

5 Extrações SQL em Janelas Longas de Data — O Padrão Watermark **8**

5.1 Estrutura da Tabela de Controle 8

5.2 “Nunca Executar em Vão” — Verificação Antes de Extrair 9

5.3 Menor Intervalo de Atualização Possível 10

6 Estratégia de Consumo em Ambiente Analítico **10**

7 Hipóteses para a Baixa Concentração de Consumidores no Norte, Nordeste e Centro-Oeste **11**

Parte 5 Documentação e Justificativas

1. Escolha e Origem dos Dados

Para responder ao desafio em tempo hábil, preservando ao máximo seus requisitos originais, optou-se pelo dataset público *Olist Brazilian E-Commerce*, disponível via API do Kaggle (kaggle.com/datasets/olistbr/brazilian-ecommerce). O dataset foi estudado diretamente na plataforma e posteriormente normalizado no Notebook 1 do repositório.

A principal vantagem dessa escolha é que a maioria das entidades exigidas no desafio — produtos, pedidos, filiais, categorias e regiões — tem correspondência direta no schema da Olist, preservando a fidelidade do problema. A única exceção foram os dados de clientes: por questões de privacidade, o dataset original não os disponibiliza com o nível de detalhe solicitado (nome, sexo, região estruturada). Esses registros foram portanto injetados sinteticamente, seguindo as distribuições demográficas e regionais realistas do Brasil, conforme descrito no Notebook 1.

2. Modelagem do Banco de Dados

2.1. Escolha do SQLite

A decisão de usar SQLite em vez de um SGBD cliente-servidor (PostgreSQL, MySQL) foi deliberada e motivada por reprodutibilidade: qualquer avaliador que clone o repositório e execute os notebooks chegará exatamente às mesmas conclusões, sem precisar configurar um servidor de banco de dados. O arquivo `varejo_gocase.db` é auto-contido e portátil.

2.2. Schema e Normalização — Terceira Forma Normal (3FN)

O banco segue a Terceira Forma Normal (3FN). Normalizar não é só burocracia acadêmica — é a diferença entre um banco que se mantém consistente ao longo do tempo e um que acumula anomalias silenciosamente. As três formas normais constroem uma sobre a outra:

1FN — Atomicidade: Cada célula deve conter um único valor indivisível. Sem listas, sem grupos repetidos dentro de uma coluna.

Errado (viola 1FN): coluna `Produtos` com valor `"Capa, Película, Fio"`. *Correto*: uma linha por produto, com `ID_Pedido` repetido.

2FN — Dependência Total da Chave: Todo atributo não-chave deve depender da chave inteira, não de parte dela. Relevante quando a chave é composta.

Errado: tabela `(ID_Pedido, ID_Produto, Nome_Produto, Quantidade)` — `Nome_Produto` depende só de `ID_Produto`. *Correto*: separar em `Itens_Pedido(ID_Pedido, ID_Produto, Quantidade)` e `Produtos(ID_Produto, Nome_Produto)`.

3FN — Sem Dependências Transitivas: Nenhum atributo não-chave pode depender de outro atributo não-chave. É a forma que o banco deste case implementa.

Errado: `Filiais(ID_Filial, Nome, Estado, Regiao)` — `Regiao` depende de `Estado`, que depende de `ID_Filial` (dependência transitiva). *Correto*: separar `Filiais(ID_Filial, Nome, ID_Estado)` e `Estados(ID_Estado, Nome_Estado, Regiao)`.

O modelo em estrela adotado — `Vendas` como tabela fato central, com chaves estrangeiras para as dimensões `Clientes`, `Filiais` e `Produtos` — é a aplicação direta da 3FN: cada entidade vive na sua própria tabela, e a tabela fato carrega apenas referências.

2.3. Escolha da Ferramenta de Visualização: Streamlit

Optou-se pelo Streamlit em vez do Power BI. Ferramentas *high-code* permitem explorar a criatividade com controle total sobre layout, lógica de filtragem, normalização de IDs e interatividade. O resultado é um dashboard com filtros globais (ano, sexo, região), explorador de filial e consulta individual de cliente, publicado em: businesscasegocase-jluca.streamlit.app.

3. Lógica das Consultas SQL

Todas as consultas foram desenvolvidas com foco em performance, com uso de índices, planos de execução (`EXPLAIN QUERY PLAN`) e recursos avançados do SQL moderno. O tema central é sempre o mesmo: *ler o mínimo possível, o mais tarde possível.*

3.1. Por que Evitar Table Scans — O Problema $O(n)$

Um *table scan* (ou *full scan*) acontece quando o banco precisa percorrer cada linha da tabela para responder a uma query. Em ciência da computação, essa operação é $O(n)$ — o tempo de execução cresce linearmente com o número de linhas. Com 1.000 linhas é imperceptível. Com 50 milhões de linhas numa tabela fato de e-commerce, é catastrófico.

Para identificar um table scan no SQLite:

```sql
EXPLAIN QUERY PLAN
SELECT * FROM Vendas WHERE Data_Venda = '2018-03-01';
-- Ruim: SCAN TABLE Vendas    <- O(n), le tudo
```

```sql
-- Bom:  SEARCH TABLE Vendas USING INDEX  <- O(log n)

-- Indice simples: evita o scan, mas ainda consulta a tabela
CREATE INDEX idx_data ON Vendas(Data_Venda);

-- Covering index: resolve a query inteiramente no indice
-- O banco nunca precisa abrir a tabela Vendas
CREATE INDEX idx_venda_cobertura
  ON Vendas(Data_Venda, ID_Produto, Valor_Total);

-- Regra pratica de construcao:
-- 1 colunas do WHERE -> 2 colunas do GROUP BY -> 3 colunas do SELECT
```

Listing 1: Índice simples vs. covering index

3.2. Consulta 1 — Faturamento Mensal por Categoria

Objetivo: calcular o faturamento total por mês e categoria no primeiro semestre de 2018. O índice funcional criado em `strftime('%Y-%m', Data_Venda)` com `Valor_Total` e `ID_Produto` incluídos permite que o `GROUP BY` seja resolvido diretamente no índice — um covering index clássico. O `EXPLAIN QUERY PLAN` confirmou o uso do índice; zero acesso à tabela fato.

```sql
SELECT
    strftime('%Y-%m', v.Data_Venda) AS Mes,
    p.Categoria,
    ROUND(SUM(v.Valor_Total), 2) AS Faturamento_Total
FROM Vendas v
INNER JOIN Produtos p ON v.ID_Produto = p.ID_Produto
WHERE v.Data_Venda BETWEEN '2018-01-01' AND '2018-06-30'
GROUP BY 1, 2
ORDER BY 1 ASC, 3 DESC;
```

Listing 2: Faturamento por mês e categoria

Por que essa métrica importa? Faturamento por categoria e mês é a base para identificar sazonalidade, detectar categorias em declínio e planejar estoque com antecedência.

3.3. Consulta 2 — Produto Mais Vendido por Filial

Objetivo: para cada filial, identificar o produto com maior volume vendido. Usa `RANK() OVER (PARTITION BY ...)` — window function que calcula o ranking de todos os produtos dentro de cada filial em uma única passagem pela tabela, eliminando a necessidade de subqueries correlacionadas que executariam $O(n)$ leituras para cada linha do resultado externo.

```sql
WITH AgregadoFilial AS (
    SELECT
        f.Nome AS Filial,
        p.Nome AS Produto,
        SUM(v.Quantidade)  AS Qtd_Total,
        SUM(v.Valor_Total) AS Receita_Total,
        RANK() OVER (
            PARTITION BY v.ID_Filial
            ORDER BY SUM(v.Quantidade) DESC
        ) AS Ranking
    FROM Vendas v
    INNER JOIN Filiais f  ON v.ID_Filial  = f.ID_Filial
    INNER JOIN Produtos p ON v.ID_Produto = p.ID_Produto
    GROUP BY v.ID_Filial, v.ID_Produto
)
SELECT Filial, Produto, Qtd_Total, ROUND(Receita_Total, 2)
FROM AgregadoFilial
WHERE Ranking = 1
ORDER BY Receita_Total DESC;
```

Listing 3: Produto mais vendido por filial com RANK()

3.4. Consulta 3 — Ticket Médio e LTV por Cliente

Objetivo: calcular frequência de compras, ticket médio e LTV por cliente. O covering index em `(ID_Cliente, Valor_Total)` permite que o SQLite calcule `AVG` e `SUM` percorrendo apenas o índice — a tabela `Vendas` não é tocada.

```sql
SELECT
    ID_Cliente,
    COUNT(ID_Venda)          AS Frequencia_Compras,
    ROUND(AVG(Valor_Total), 2) AS Ticket_Medio,
    ROUND(SUM(Valor_Total), 2) AS Valor_Total_Gasto
FROM Vendas
GROUP BY ID_Cliente
```

```
ORDER BY Valor_Total_Gasto DESC
LIMIT 10;
```

Listing 4: Ticket médio e LTV

Por que essas métricas importam? Ticket médio e LTV são os dois indicadores centrais de valor de cliente. Juntos, permitem segmentar a base em estratégias de retenção e up-sell diferenciadas.

3.5. Consulta 4 — Clientes Fiéis por Região

Objetivo: identificar clientes com mais de 3 compras, agrupados por região. O filtro `HAVING` é aplicado após o `GROUP BY` e otimizado pelo índice de `ID_Cliente`. O critério de fidelidade foi parametrizado como variável, facilitando análises de sensibilidade.

```
SELECT
    c.Regiao,
    c.Nome AS Cliente,
    COUNT(v.ID_Venda) AS Total_Compras
FROM Vendas v
INNER JOIN Clientes c ON v.ID_Cliente = c.ID_Cliente
GROUP BY v.ID_Cliente
HAVING Total_Compras > 3
ORDER BY Total_Compras;
```

Listing 5: Clientes fiéis com HAVING

3.6. Consulta 5 — Tendências Regionais por Mês

Objetivo: calcular variação percentual de vendas mês a mês por região. Usa CTE duplo — a primeira CTE agrega (reduz de milhões para centenas de linhas), a segunda aplica `LAG()` sobre esse conjunto já reduzido. Nunca se aplica window functions sobre tabelas fato completas quando é possível agregar antes: isso transforma um $O(n)$ em $O(k)$, onde k é o número de meses $\times$ regiões.

```
WITH VendasMensais AS (
    SELECT
        c.Regiao,
        strftime('%Y-%m', v.Data_Venda) AS Mes,
        SUM(v.Valor_Total) AS Venda_Atual
    FROM Vendas v
    INNER JOIN Clientes c ON v.ID_Cliente = c.ID_Cliente
    WHERE v.Data_Venda >= '2018-01-01'
    GROUP BY c.Regiao, Mes
```

```sql
),
CalculoTendencia AS (
    SELECT
        Regiao, Mes,
        ROUND(Venda_Atual, 2) AS Venda_Atual,
        ROUND(LAG(Venda_Atual) OVER
            (PARTITION BY Regiao ORDER BY Mes), 2)
            AS Venda_Mes_Anterior,
        ROUND(((Venda_Atual / LAG(Venda_Atual) OVER
            (PARTITION BY Regiao ORDER BY Mes)) - 1) * 100,
               2)
            AS Crescimento_Pct,
        ROW_NUMBER() OVER
            (PARTITION BY Regiao ORDER BY Mes) AS Mes_Seq
    FROM VendasMensais
)
SELECT Regiao, Mes, Venda_Atual, Venda_Mes_Anterior,
   Crescimento_Pct
FROM CalculoTendencia
WHERE Venda_Mes_Anterior IS NOT NULL AND Mes_Seq <= 5
ORDER BY Regiao, Mes;
```

Listing 6: Tendência com LAG() e CTE duplo

Parte 6 Requisitos Adicionais

4. Google Sheets com JavaScript — Gestão de Limites e Quotas

4.1. Controle de Rate Limiting com Back-off Exponencial

Quando uma API retorna erro 429 (*Too Many Requests*), a pior resposta possível é tentar novamente imediatamente — você continua tomando erro e esgota a quota restante em segundos. O *back-off* exponencial resolve isso de forma elegante: cada tentativa fracassada aumenta o tempo de espera geometricamente, com um teto máximo para evitar esperas infinitas.

4.2. Batching — Nunca Uma Requisição por Célula

O padrão mais comum de desperdício de quota em Apps Script é fazer uma chamada à planilha por linha ou por célula dentro de um loop. A API do Google aceita operações em blocoç.

4.3. Paralelismo Controlado e Caching

Para chamadas a APIs externas dentro do Apps Script, `Promise.allSettled()` com um semáforo de concorrência máxima (ex.: 5 chamadas simultâneas) evita que o paralelismo irrestrito estoure a quota em segundos. Para dados que não mudam com frequência, o `CacheService` do Google Apps Script armazena resultados com TTL definido pela frequência real de atualização da fonte.

5. Extrações SQL em Janelas Longas de Data — O Padrão Watermark

Para janelas longas de extração (ex.: 3 anos de dados transacionais), nunca se executa uma query monolítica. O padrão correto é a extração incremental com *watermark*: um bookmark persistente que registra exatamente até onde a extração foi bem-sucedida, permitindo retomar de qualquer ponto sem reprocessar dados já capturados.

5.1. Estrutura da Tabela de Controle

```sql
CREATE TABLE watermark (
    fonte         TEXT      PRIMARY KEY,
    ultima_data   TIMESTAMP,    -- ate onde foi com sucesso
    status        TEXT,         -- ok | running | failed
    atualizado_em TIMESTAMP
```

```
);

INSERT INTO watermark VALUES (
    'vendas_olist', '2018-01-01␣00:00:00', 'ok',
       CURRENT_TIMESTAMP
);
```

Listing 7: Tabela de controle de watermark

5.2. "Nunca Executar em Vão" — Verificação Antes de Extrair

Imagine uma extração agendada a cada 10 minutos. Das 23h às 6h, nenhuma venda ocorre. Sem verificação prévia, o processo conecta ao banco, lê `MAX()`, compara, extrai zero linhas e encerra — 42 vezes, consumindo conexões, CPU e eventual custo de cloud sem nenhum dado novo. "Nunca executar em vão" é simplesmente checar se existe trabalho real antes de fazer qualquer coisa pesada:

```
max_fonte = busca_max_data(conn_fonte)
if max_fonte <= watermark:
    return  # encerra aqui, zero custo adicional

# so chega aqui se houver dados genuinamente novos
extrair(...)
```

Listing 8: Guarda de execução vã

Há um detalhe importante: o próprio `SELECT MAX()` pode ser caro se a tabela não tem índice em `Data_Venda`. Numa tabela com 50 milhões de linhas, essa verificação "inocente"faz um full scan $O(n)$ só para decidir se vai executar. A solução é manter uma tabela de metadados na fonte, atualizada por trigger, que armazena o último timestamp de escrita em uma única linha — a verificação vira $O(1)$.

```
CREATE TABLE metadados_extracao (
    tabela        TEXT PRIMARY KEY,
    ultima_escrita TIMESTAMP
);

-- Trigger que mantem o metadado atualizado automaticamente
CREATE TRIGGER atualiza_meta
AFTER INSERT ON Vendas
BEGIN
    INSERT OR REPLACE INTO metadados_extracao
    VALUES ('Vendas', NEW.Data_Venda);
```

```sql
END;

-- Verificacao agora custa microsegundos, nao segundos
SELECT ultima_escrita FROM metadados_extracao
WHERE tabela = 'Vendas';
```

Listing 9: Metadados para verificação $O(1)$

O detalhe mais crítico do fluxo acima: o watermark só avança após o `commit` no destino ter sido concluído. Se o processo cair no passo 6, na próxima execução o `status` estará como `running` — sinal claro de falha anterior — e a extração retoma do mesmo ponto sem perda de dados.

5.3. Menor Intervalo de Atualização Possível

O intervalo mínimo é ditado pela latência de chegada dos dados na fonte: se a operação gera dados com delay de 15 minutos (ex.: fila de processamento de pedidos), rodar a extração a cada 5 minutos gera apenas leituras vazias e carga desnecessária. A lógica correta: extrair somente quando `MAX(Data_Venda)` na fonte for maior que o watermark armazenado, respeitando a folga de segurança para transações em aberto.

6. Estratégia de Consumo em Ambiente Analítico

A decisão entre conexão direta, camada intermediária e materialização depende de três variáveis: volume, frequência de atualização e complexidade das transformações.

Tabela 1: Comparativo de estratégias analíticas

Estratégia	**Quando Usar**	**Exemplo**
Conexão direta ao banco	Volume pequeno, queries simples, acesso esporádico	SQLite + Streamlit com `@st.cache_data`
Data Warehouse / Staging	Volume grande, múltiplas fontes, histórico e auditoria	BigQuery, Redshift, DuckDB
Views Materializadas	Métricas caras (LTV, coortes), SLA < 2 segundos	`CREATE MATERIALIZED VIEW` no PostgreSQL

No contexto deste case, usando SQLite + Streamlit, a conexão é direta ao banco com cache em memória (`@st.cache_data`), o que equivale funcionalmente a uma materialização leve: os dados são carregados uma vez e servidos em memória para todas as interações do usuário, sem reprocessamento a cada clique.

7. Hipóteses para a Baixa Concentração de Consumidores no Norte, Nordeste e Centro-Oeste

O dashboard evidencia uma concentração expressiva de vendas no eixo Sudeste-Sul, com Norte, Nordeste e Centro-Oeste respondendo por uma fatia proporcionalmente pequena — bem abaixo do que o peso populacional dessas regiões sugeriria. As hipóteses abaixo não são mutuamente exclusivas; na prática, todas operam simultaneamente.

H1 — Logística e Custo de Frete

O Brasil tem uma das maiores disparidades logísticas do mundo. Entregar um produto a Manaus (AM) ou Macapá (AP) pode custar 3x a 5x mais do que entregar em São Paulo, e o prazo pode ultrapassar 15 dias úteis. Para um consumidor nordestino ou nortista, o frete frequentemente supera o valor do produto — especialmente nas categorias de ticket médio baixo que dominam o catálogo de e-commerce popular. O resultado é abandono de carrinho antes mesmo de chegar ao checkout. Apesar disso ser verdade, pode explicar uma parte do efeito observado. Não explicaria todo pois outros e-commerce (mercado livre, Amazon) fazem muito sucesso e têm muitos consumidores nessas regiões.

No entanto, A Olist opera via marketplace com vendedores concentrados em SP, MG e RS. A maioria dos vendedores não tem centros de distribuição fora do Sudeste, o que agrava estruturalmente o custo logístico para as demais regiões.

H2 — Viés de Origem do Dataset

Considere a hipótese a seguir como uma expansão de H1...

Vale considerar um viés estrutural do próprio dataset: a Olist é uma empresa fundada em Curitiba (PR), com operações e base de vendedores historicamente concentradas no Sul e Sudeste. Os dados refletem não só o comportamento dos consumidores, mas também a capacidade de oferta da plataforma naquele período. Um consumidor do Ceará que tentou comprar e não encontrou vendedor com entrega viável simplesmente não gerou uma transação — e portanto não aparece nos dados. Como vimos no dashboard, a evolução logintudinal das vendas mostra que a operação é nova e está em expansão. O que provavelmente acontece com este E-Commerce em específico é que eles estão expandindo sua malha de distribuição e isso ocorre exatamente em regiões próximas ao centro operacional da empresa. Devido a isso, o foco inicial é no sul/sudeste e depois vai se alastrando pelo resto do Brasil, i.e, o que o padrão longitudinal demonstra.

Contudo, é necessário acesso a dados internos da própria empresa, incluindo relatórios, para confirmar isso.

H3 — Centro-Oeste: o Caso Específico

O Centro-Oeste merece análise separada: tem renda média mais alta que o Nordeste e infraestrutura razoável, mas é uma região de baixíssima densidade demográfica fora do eixo Brasília-Goiânia. O volume naturalmente menor de consumidores — não necessariamente de renda ou acesso — explica uma parte da sub-representação; vale observar que isso ocorre também em nos outros E-commerces já citados...

Brasília, que concentra a renda da região, provavelmente responde pela maior parte das transações do Centro-Oeste no dataset. Veja

Síntese: a sub-representação de Norte, Nordeste e Centro-Oeste no dashboard é multicausal. Logística é a hipótese mais forte e mais acionável — reduzir o custo de frete para essas regiões (via centros de distribuição regionais ou subsídio logístico) teria impacto direto e mensurável no volume de transações. As demais hipóteses atuam como amplificadores do mesmo efeito.